令 和 3 年 度

卒 業 研 究 報 告 書

機械学習によるMOSFETの電流電圧特性の予測

指導教官 木場　隼介
報告者　　杉本　英

神戸市立工業高等専門学校
電子工学科

（論文要旨）

トランジスタの高集積化と高性能化は、電子情報機器の性能向上、ひいては情報化社会を牽引してきた。しかし、トランジスタを小型化することでトンネル効果によりサブスレッショルド特性が劣化してしまう。また、ゲートと基板の間の容量結合を一定に保つため、ゲート酸化膜を薄くすると、それによっても弊害が発生する。すなわち、ゲートリーク電流の増大による消費電力の増加が生じる。そこで、ゲートの誘電体を SiO_2 から別の材料に変えることで、ゲートの制御性を維持したままゲートリーク電流をおさえることができる。そのため、材料を変えることで一体どの程度結果が得られるのかあらかじめ結果の概要を知ることで効率よく作成できるようになることが望まれる。しかし、材料が変わると誘電率だけでなく、界面の平滑さや欠陥密度が大きく変わり、Si の式ではうまく予測できない。そのため、コンパクトモデルを作る必要があるが、人的負担が大きく、材料が変わるたびに再度作る必要がある。そこで、機械学習を用いることでその負担を軽減できると期待できる。デバイス開発のデータを用いて機械学習の教師あり学習を使用することで、材料等を変えたときの概要を V_G–V_D–I_D 予測することができる。本研究では、デバイスにおいて重要な変数である V_D(ドレイン電圧), V_G(ゲート電圧), I_D(ドレイン電流) を用いてどのような V_G–V_D–I_D 特性が得られるかを予測することを目的として、機械学習のモデルを作成した。

結果として、仮想データや論文から得られたデータから作成したモデルを使用した予測では、サブスレッショルド特性の予測に難はあるものの、許容範囲内の誤差で収まっており、良い結果が得られたと考えられる。また、それらのモデルを使用した予測では、I_D の V_D 依存性を正しく表現するとともに、V_G が増えると、I_D が上昇する非線形的変化を正しく学習できていると推測できる。最後に、学習モデルの補間の正確さは、一定量情報が減ると精度は下がるものの、そこからは精度があまり減少しないことから、試作したデバイスから得られた情報が少なくてもある程度の予測精度を持ったモデルを作成できることが分かった。つまり、測定者が大量のデータをとる必要がなくなるため、デバイスやコンパクトモデルの作製者だけでなく、測定者に対しても負担が軽減できることが期待できる。

今後の課題としては、まず、精度の向上があげられる。とくに、デバイスは大量に試作品を作ることが困難であるため、必然的にデータ量は少なくなる。そのため、データ量が少なくても精度が落ちないようにする必要がある。グリッドサーチと別の手法を一緒に使用してパラメータの調整をしたり、全く新しいカーネルを開発するなどすれば精度が向上できると考えられる。次に、サブスレッショルド特性の予測についてである。現状、どのパラメータでも改善しなかったため、SVR を応用した回帰アルゴリズムを作成することが、最善策だと予想される。ゲート酸化膜やチャネル長や誘電率などの説明変数の追加である。本研究では、V_G と V_D, I_D しか使用しなかったが、誘電率やゲート酸化膜などを使用することで、未知のデバイスに対しての予測が可能になりうる。

目 次

第 1 章　はじめに **1**

第 2 章　理論 **2**

2.1 scikit-learn 3

2.2 SVR(Support Vector Regression) 5

2.2.1 SVM(Support Vector Machine) について 5

2.2.2 SVR(Support Vector Regression) 7

2.3 カーネル関数 10

2.3.1 rbf(Radial basis function kernel) カーネル 10

2.3.2 線形カーネル 10

2.3.3 多項式カーネル 10

2.4 各種パラメータについてとグリッドサーチについて 11

2.4.1 パラメータについて 11

2.4.2 γ と C 関係性について 12

2.4.3 グリッドサーチについて 12

第 3 章　実装方法 **13**

3.1 訓練データとテストデータのラベル付け 13

3.2 グリッドサーチによるパラメータの調整 13

3.3 SVR によるモデルの作成 14

第 4 章　実験結果 **15**

4.1 仮想データを用いた予測精度の確認 15

4.2 訓練用 csv ファイルの作成 16

4.3 元データと学習したモデルの比較 16

4.4 ゲート電圧を変化させたときの比較 17

4.5 学習モデルの補間の正確さの確認 18

第 5 章　考察 **20**

5.1 仮想データを用いた予測精度の確認 20

5.2 元データと学習したモデルの比較について 20

5.3 ゲート電圧を変化させたときの比較について 20

5.4 学習モデルの補間の正確さの確認について 20

第 6 章　結論と今後の課題 **21**

6.1 結論 21

6.2 今後の課題 21

参考文献 **22**

謝辞 **23**

第1章　はじめに

トランジスタの高集積化と高性能化は、電子情報機器の性能向上、ひいては情報化社会を牽引してきた。しかし、トランジスタを小型化することでトンネル効果によりサブスレッショルド特性が劣化してしまう。また、ゲートと基板の間の容量結合を一定に保つため、 すると、それによっても弊害が発生する。すなわち、ゲートリーク電流の増大による消費電力の増加が生じる。そこで、ゲートの誘電体を SiO_2 から別の材料に変えることで、ゲートの制御性を維持したままゲートリーク電流をおさえることができる。そのため、材料を変えることで一体どの程度結果が得られるのかあらかじめ結果の概要を知ることで効率よく作成できるようになることが望まれる。しかし、材料が変わると誘電率だけでなく、界面の平滑さや欠陥密度が大きく変わり、Si の式ではうまく予測できない。そのため、コンパクトモデルを作る必要があるが、人的負担が大きく、材料が変わるたびに再度作る必要がある。そこで、機械学習を用いることでその負担を軽減できると期待できる。デバイス開発のデータを用いて機械学習の教師あり学習を使用することで、材料等を変えたときの概要を V_G–V_D–I_D 予測することができる。本研究では、デバイスにおいて重要な変数である V_D(ドレイン電圧), V_G(ゲート電圧), I_D(ドレイン電流) を用いてどのような V_G–V_D–I_D 特性が得られるかを予測することを目的とする。

第2章　理論

本研究で使用した手法を説明する前に従来から用いられてきた性能予測法であるモンテカルロ法を用いたデバイスシミュレーションやコンパクトモデルについて述べる。

モンテカルロ法は、デバイスシミュレーションに用いられる手法である。性能予測の精度は非常に高いが、確率平均をとるため、数時間〜数日間に及ぶ膨大な計算コストがかかる。

また、コンパクトモデルは回路シミュレーションに用いられることが多いが、近年のデバイスに関する知見も織り込んだモデル構築が盛んである。計算コストは低いためモンテカルロ法より時間はかからないが、同一モデルは同一の材料にしか適用できず、同一モデルが適用できるデバイス構造やサイズも限定される。よって、新しいデバイスに対してはモデルの構築が 1 からする必要がある。

このように、従来の方法では計算コストや汎用性に難があった。それに対して、機械学習による予測では計算コストを数十分〜数時間に抑えることができ、同一の手法で幅広い構造やデバイスについて、少量の情報から正確な性能予測を可能にすることができる。つまり、計算コストはモンテカルロ法より抑えつつ、汎用性はコンパクトモデルよりも高い手法として利用することができる。

次に、本研究で使用した機械学習のライブラリである scikit-learn 以外の機械学習ライブラリの TensorFlow や Pytorch について説明する。

TensorFlow は、2015 年に公開されたオープンソースのディープラーニング・フレームワークで、2017 年に正式版がリリースされた。主要な OS や複数のプログラミング言語に対応しており汎用性の高さが強みである。また、TensorFlow は利用者が多いのが特徴で、書籍やネット上の記事など学習教材が豊富に揃っている。

Pytorch は、2016 年に公開されたオープンソースのディープラーニング・フレームワークである。一般の普及率では TensorFlow に劣るが、コードの作成やモデルの操作が直感的で分かりやすく、主に研究の分野で使用されている。

この 2 つのライブラリも非常に分かりやすく使い勝手の良いライブラリではあるが、公式サイトから得られる情報やデータの前処理を行うライブラリを持つ観点から scikit-learn を採用した。

2.1 scikit-learn

scikit-learn は、python の機械学習のライブラリでありオープンソースである。Numpy、SciPy、matplotlib といった Python のライブラリ上で動作し、scikit-learn を使えば、誰でも手軽に機械学習の前処理やモデリング、評価指標の算出などを行うことができる。具体的に 3 つの利点を示す。

1) アルゴリズムが豊富かつ使いやすい。
   機械学習のアルゴリズムだけでなく、使用するデータの前処理等のアルゴリズムも組み込まれているので非常に豊富である。

2) アルゴリズムチートシートを参考に、簡単に適切な手法を選択できる。
   どのアルゴリズムを使えばよいか、チートシート呼ばれるフローチャートを用いることで適したアルゴリズムを見つけることができる。

3) 公式サイトの解説が丁寧で便利である。
   関数の引数の説明の他、サンプルプログラムなども掲示されているため、非常に分かりやすい。

scikit-learn の主な機能について説明する。

1) classification(分類)
   分類はデータを学習して、新たな入力に対する出力のラベルを予測する手法である。回帰と同様に気象に関して例えるならば、過去の気象データと位置情報を学習し、その地点の翌日の天気（晴れ、曇り、雨、雪）を予測するのは、分類である。

2) regression(回帰)
   回帰とは、データセットを学習して新たな入力に対する出力の数値を予測する手法のことである。例えば、過去の気象データと位置情報を学習し、その地点の翌日の気温を予測する際には回帰を行う。

3) clustering(クラスタリング)
   クラスタリングは、教師なし学習のアルゴリズムであり、データの特徴から関連の深いデータや似通ったデータを見つけ、クラスターに分ける（＝グルーピングする）手法である。例えば、企業の購買データの分析などに応用されていて、クラスタリングの結果に基づいて企画戦略をすることもある。scikit-learn では、クラスタリングも K-means をはじめ数多くのアルゴリズムで実行可能である。

4) dimensionality reduction(次元削除)
   次元削減は教師なし学習の一種である。データセットの特徴量同士の関係を学習し、より簡潔かつ効率的に特徴量の情報を表現することなどを目的に、新たな特徴量を計算で導出して特徴量の数を削減する手法である。例えば、近年盛んに様々なメディアで取り上げられているビッグデータを処理する際に、データセットのあまりの大きさにコンピュータの処理性能が不足することがある。そういった場合に、次元削減をすることでコンピュータの性能不足を補うことができる。scikit-learn では、主成分分析や因子分析などで次元削減を行うことができる。

次に、scikit-learn で使用するチートシートについて説明する。scikit-learn のチートシートを Figure2.1 に示す。

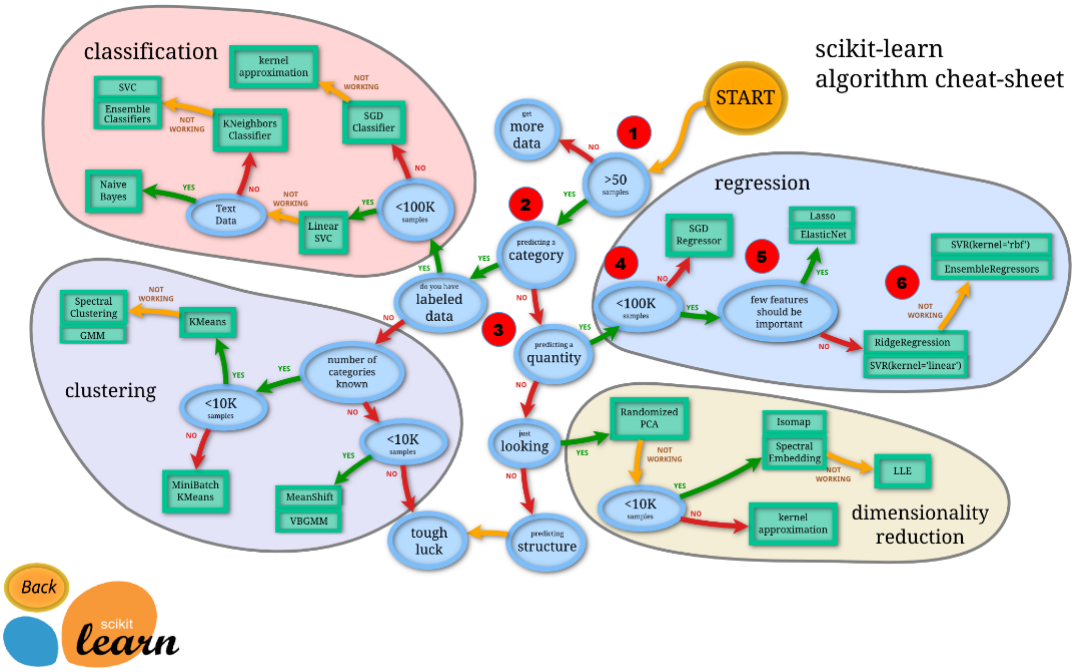

Figure 2.1 scikit-learn のチートシート [3]

それぞれの分岐部分について説明する。①では、データ数が 50 を下回っているか確認している。下回っている場合は、再度データを収集しなければならない。なお、本研究では 50 を超えているため②へ進む。②では、説明変数や目的変数がカテゴリーを持つかの確認している。例えば、天気 (晴れ、曇り、雨、雪) などはカテゴリーとなる。本研究では、すべての変数が数値であるため、カテゴリを持たない。③では、数値の予測をするかどうかの確認である。本研究では、V_D, V_G などの数値である説明変数から目的変数である I_D の数値を予測したいため、④へ進む。④では、サンプル数が１０万件以下かどうか確認している。本研究では、下回っているため④へ進む。⑤では、説明変数がすべて重要かどうか確認している。本研究で入力する説明説明変数は、V_D, V_G など目的変数である I_D に対して、すべて需要であるため SVR(liner) へ進む。括弧内は、使うカーネル関数を示しており、カーネル関数については、節 2.3 にて説明する。liner では、線形に対しての学習のモデルしか作成できない。V_D–V_G–I_D は非線形な特性であるため、⑥へ進む。ここで、本研究で使用する SVR(rbf) となる。

2.2 SVR(Support Vector Regression)

SVR は、SVM(サポートベクターマシン) を回帰分析に応用したものである。SVR の説明の前に SVM について少し説明する。

2.2.1 SVM(Support Vector Machine) について

SVM(Support Vector Machine) は、線形判別関数を用いたクラス分類のことである。二つのクラスがあったとき、どちらに属するか決めるものである。以下に線形判別関数についてイメージ図を示す。クラス 1 を＋1 の集合とし、クラス 2 を-1 の集合とする。また、線形判別関数は、式 (2.1) で表すことができる。

$$f\left(x_1, x_2\right) = w_1 x_1 + w_2 x_2 + b = \boldsymbol{x}\boldsymbol{w} + b \ , \boldsymbol{x} = \left[x_1, x_2\right] \ , \boldsymbol{w} = \begin{bmatrix} w_1 \\ w_2 \end{bmatrix} \tag{2.1}$$

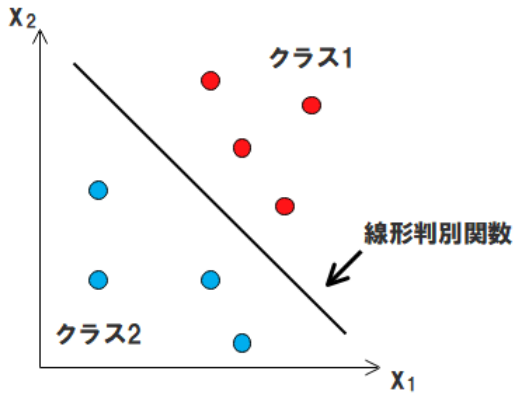

Figure 2.2 線形判別関数のイメージ図

スラック変数の導入

クラスの分布によってはきれいに判別できない場合がある。この場合は、スラック変数 ξ を導入する。以下にスラック変数のイメージ図を示す。サンプルごとの ξ_i の和 $(\sum_{i=1}^{n} \xi_i)$ を最小化することで判別を可能にする。なお、n は総サンプル数である。

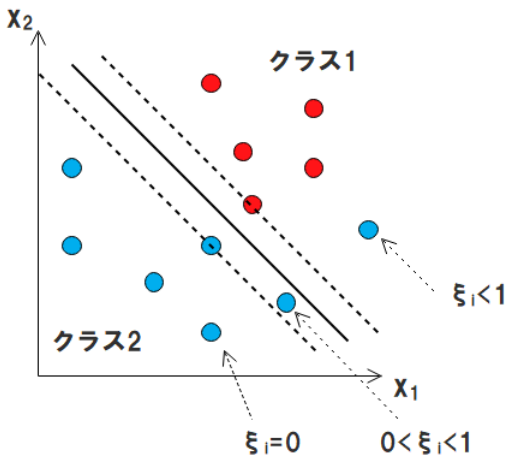

Figure 2.3 スラック変数導入のイメージ図

マージンの最大化

SVM では、マージン (点と直線との距離) を最大化するように判別関数を決める。マージンの計算式とイメージ図を以下に示す。式 (2.2) を最大化するか式 (2.3) を最小化する必要がある。

$$margin = \frac{2}{||\boldsymbol{w}||} = \frac{2}{\sqrt{w_1^2 + w_2^2}} \tag{2.2}$$

$$margin = \frac{1}{2}||\boldsymbol{w}|| \tag{2.3}$$

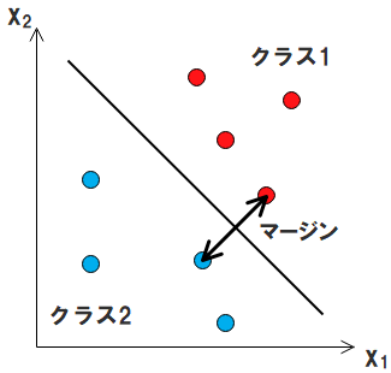

Figure 2.4 マージンのイメージ図

以上のことから $xi_i(\sum_{i=1}^{n} \xi_i)$ の和と式 (2.3) との和を最小化すればよい。以下に最小化しなければならない式を示す。ただし、計算の都合上 $\frac{1}{2}||\boldsymbol{w}|| \to \frac{1}{2}||\boldsymbol{w}||^2$ としている。

$$\frac{1}{2}||\boldsymbol{w}||^2 + \boldsymbol{C}\sum_{i=1}^{n} \xi_i \tag{2.4}$$

ただし、$\xi_i \geq 0, \quad y^(i) f\left(x^(i)\right) \geq 1 - \xi_i$

$\boldsymbol{C}$: 2 つの項のバランスを決める係数

$x^{(i)}$: i 番目のサンプルの説明変数

$y^{(i)}$: i 番目のサンプルの値 (1 もしくは-1)

上記の式を最小化すればよい。次に、SVR について解説する。

2.2.2 SVR(Support Vector Regression)

まず、線形の回帰分析手法の式を示す。

$$y = x_1 b_1 + x_2 b_1 + f \tag{2.5}$$

y: 目的変数, x: 説明変数, b: 回帰係数, f: y の x で表すことができない部分 (誤差、残差)

と表される。次に、回帰係数 $\boldsymbol{b}$ を

$$\boldsymbol{b} = [b_1, b_2, \cdot\cdot\cdot, b_m]$$

m: 説明変数の数, b: 回帰係数

と置いたとき、あるサンプル (i 番目のサンプル) の目的変数の推定値 $f\left(\boldsymbol{x}^{(i)}\right)$ は、以下で表される。

$$\text{線形回帰モデル} : f\left(\boldsymbol{x}^{(i)}\right) = \boldsymbol{x}^{(i)}\boldsymbol{b} \tag{2.6}$$

次に、説明変数を高次元空間への写像 (非線形写像) すると、

$$\boldsymbol{x} \to \phi(\boldsymbol{x}) \tag{2.7}$$

になるため、式 (2.6) は

$$f\left(\boldsymbol{x}^{(i)}\right) = \phi\left(\boldsymbol{x}^{(\boldsymbol{i})}\right)\boldsymbol{w} + c \tag{2.8}$$

$$\boldsymbol{w} = [w_1, w_2, \cdot\cdot\cdot, w_k]$$

c: 定数項 , w_i: 重み , k: 高次元空間での次元数

マージンの最大化

SVM のように最小化が必要な式を以下に示す。

$$\frac{1}{2}||\boldsymbol{w}||^2 + \boldsymbol{C}\sum_{i=1}^{n} h\left(y^{(i)} - f\left(\boldsymbol{x}^{(i)}\right)\right) \tag{2.9}$$

C: 2 つの項のバランスを決める係数

h: 誤差関数

n: モデル構築用サンプル数

で表せられる。SVM では式 (2.4) を最小化する必要があったが、SVR では式 (2.9) である。

SVR の誤差関数

誤差関数について定義式とイメージ図を示す。

$$h\left(y^{(i)} - f\left(\boldsymbol{x}^{(i)}\right)\right) = \max\left(0, |y^{(i)} - f\left(\boldsymbol{x}^{(i)}\right)| - \varepsilon\right) \tag{2.10}$$

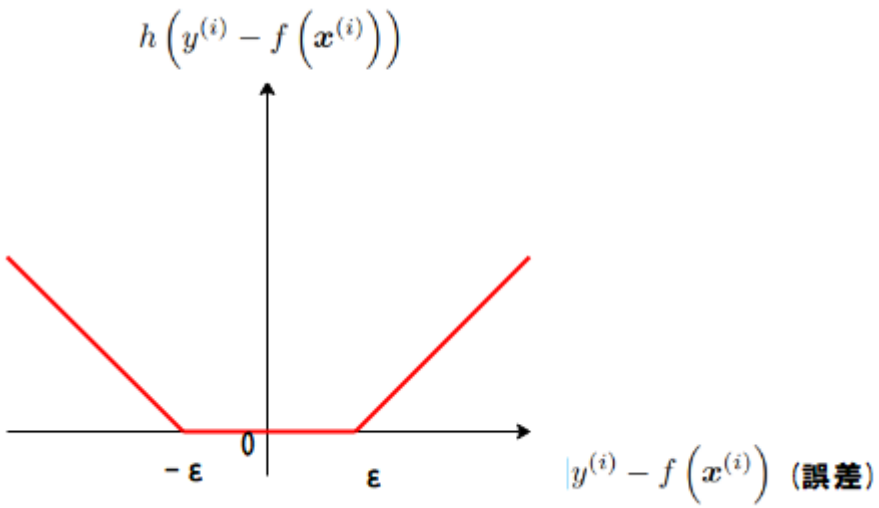

Figure 2.5 誤差関数のイメージ図

この ε は許容する誤差を決めており、$-\varepsilon \leq$ 誤差 $\leq \varepsilon$ のとき、誤差 $= 0$ として、扱う。

スラック変数の導入

SVM と同様にスラック変数 ξ, ξ^* を導入する。以下に定義式を示す。

$$y^{(i)} \leq f\left(\boldsymbol{x}^{(i)}\right) + \varepsilon + \xi_i \tag{2.11}$$

$$y^{(i)} \geq f\left(\boldsymbol{x}^{(i)}\right) - \varepsilon - \xi_i \tag{2.12}$$

この二つの式から式 (2.9) を以下のように変形できる。

$$\frac{1}{2}||\boldsymbol{w}||^2 + \boldsymbol{C}\sum_{i=1}^{n} h\left(y^{(i)} - f\left(\boldsymbol{x}^{(i)}\right)\right) = \frac{1}{2}||\boldsymbol{w}||^2 + \boldsymbol{C}\sum_{i=1}^{n}(\xi_i + \xi_i^*) \tag{2.13}$$

以上のように誤差だけでなく、回帰係数も一緒に最小化することで過学習を防ぐことができる。

ラグランジュ乗数の導入

また、ラグランジュ乗数、$a_i, a_i^*, \beta_i, \beta_i^*(i$ =1,2,...,$n)$ を式 (2.13) に導入する。導入した式を以下に示す。

$$G = C\sum_{i=1}^{n}(\xi_i+\xi_i^*)+\frac{1}{2}||\boldsymbol{w}||^2-\sum_{i=1}^{n}(\beta_i\xi_i+\beta_i^*\xi_i^*)-\sum_{i=1}^{n}a_i\left(\varepsilon+\xi_i+f\left(\boldsymbol{x}^{(i)}\right)-y_i\right)-\sum_{i=1}^{n}a_i^*\left(\varepsilon+\xi_i^*-f\left(\boldsymbol{x}^{(i)}\right)+y_i\right) \tag{2.14}$$

次に G を $\boldsymbol{w}, c, \xi_i, \xi_i^*$ それぞれで偏微分して 0 にする。G を各変数で偏微分して 0 したときの式を以下に示す。

$$\boldsymbol{w} = \sum_{i=1}^{n}(a_i - a_i^*)\phi\left(\boldsymbol{x}^{(i)}\right)^T \ (G \text{ を } \boldsymbol{w} \text{ で偏微分して } 0) \tag{2.15}$$

$$\sum_{i=1}^{n}(a_i - a_i^*) = 0 \ (G \text{ を } c \text{ で偏微分して } 0) \tag{2.16}$$

$$a_i + \beta_i = C \ , \ i = 1, 2, ..., n (G \text{ を} \xi_i \text{で偏微分して } 0) \tag{2.17}$$

$$a_i^* + \beta_i^* = C \ , \ i = 1, 2, ..., n (G \text{ を} \xi_i^* \text{で偏微分して } 0) \tag{2.18}$$

以上の 4 つの式を使って式 (2.14) を変形すると、

$$G = -\frac{1}{2}\sum_{i=1}^{n}\sum_{j=1}^{n}(a_i - a_j^*)(a_j - a_j^*)K\left(\boldsymbol{x}^{(i)}, \boldsymbol{x}^{(i)}\right) - \varepsilon\sum_{i=1}^{n}(a_i + a_i^*) + \sum_{i=1}^{n}(a_i - a_i^*)y_i \tag{2.19}$$

$$K\text{: カーネル関数 } K\left(\boldsymbol{x}^{(i)}, \boldsymbol{x}^{(j)}\right) = \phi\left(\boldsymbol{x}^{(i)}\right)\phi\left(\boldsymbol{x}^{(j)}\right)^T$$

カーネル関数は 2.3 で解説する。

2 次計画問題による a_i, a_i^* の導出

$a_i, a_i^*, \beta_i, \beta_i^*$ は、Lagrange 定数であり、$a_i \geq 0, a_i^* \geq 0, \beta_i \geq 0, \beta_i^* \geq 0$ である。式 (2.17) と式 ((2.18) より、$a_i \geq C, a_i^* \geq C$ である。以上より G を a_i, a_i^* に対して最大化する 2 次計画問題を解くと、a_i, a_i^* が求まる。制約を以下に示す。

$$\sum_{i=1}^{n}(a_i - a_i^*) = 0 \ , \ 0 \geq a_i \geq C \ , \ 0 \geq a_i^* \geq C \tag{2.20}$$

SVR の回帰式の導出

式 (2.9) に式 (2.16) を代入することで、以下の式を得ることができる。

$$f\left(\boldsymbol{x}^{(j)}\right) = \sum_{i=1}^{n}(a_i - a_i^*)\phi\left(\boldsymbol{x}^{(j)}\right)\phi\left(\boldsymbol{x}^{(j)}\right)^T + c \tag{2.21}$$

$$a_i \ , \ a_i^* \text{ はラグランジュ乗数であり求められる}$$

が導出できる。
また、式 (2.20) にて定義したカーネル関数を用いると、以下の式を得る。

$$f\left(\boldsymbol{x}^{(j)}\right) = \sum_{i=1}^{n}(a_i - a_i^*)K\left(\boldsymbol{x}^{(i)}, \boldsymbol{x}^{(j)}\right) + c \tag{2.22}$$

定数項の算出

定数項である c の算出をする。誤差は誤差関数のときに決めた値 (誤差 $= \pm\varepsilon$) である。これを ε チューブと呼ぶ。ε チューブ上のサンプルで c を計算し、それらの平均値を最終的な c として使用する。

$$c = y^{(i)} - \sum_{i=1}^{n}(a_i - a_i^*)K\left(\boldsymbol{x}^{(i)}, \boldsymbol{x}^{(j)}\right) - \varepsilon \tag{2.23}$$

$$c = y^{(i)} - \sum_{i=1}^{n}(a_i - a_i^*)K\left(\boldsymbol{x}^{(i)}, \boldsymbol{x}^{(j)}\right) + \varepsilon \tag{2.24}$$

2.3 カーネル関数

以下の式のような線形判別関数を

$$f(x) = \sum_{i=1}^{n} a_i y^{(i)} \boldsymbol{x} \boldsymbol{x}^{(i)T} + b \tag{2.25}$$

高次元空間へ写像 (式 (2.4)) すると以下のようになる。

$$f(x) = \sum_{i=1}^{n} a_i y^{(i)} \phi\left(\boldsymbol{x}\right) \phi\left(\boldsymbol{x}^{(i)}\right)^T + b \tag{2.26}$$

この場合、$\phi\left(\boldsymbol{x}\right)$ を求めるのではなく $\phi\left(\boldsymbol{x}\right) \phi\left(\boldsymbol{x}^{(i)}\right)^T$ を求めればよい。よって、式 (2.9) のようなカーネル関数を作る。

2.3.1 rbf(Radial basis function kernel) カーネル

以下に、今回使用したカーネル関数を示す。

$$K\left(\boldsymbol{x}^{(i)}, \boldsymbol{x}^{(j)}\right) = \exp\left(-\frac{||\boldsymbol{x}^{(i)} - \boldsymbol{x}^{(j)}||^2}{2\sigma^2}\right) = \exp\left(-\gamma||\boldsymbol{x}^{(i)} - \boldsymbol{x}^{(j)}||^2\right) \tag{2.27}$$

この関数は rbf(Radial basis function kernel) カーネルまたはガウシアンカーネルとも呼ばれ、複雑な分類を可能にする。式中の $||\boldsymbol{x}^{(i)} - \boldsymbol{x}^{(j)}||$ は、サンプル間のユークリッド距離を示しおり、距離が近いほどそのサンプルたちは類似度が高いと言える。

2.3.2 線形カーネル

今回、使用を見送ったカーネルについて説明する。

$$K\left(\boldsymbol{x}^{(i)}, \boldsymbol{x}^{(j)}\right) = \boldsymbol{x}^{(i)}, \boldsymbol{x}^{(j)T} \tag{2.28}$$

この関数は線形カーネルと呼ばれるカーネル関数である。MOSFET の電流ー電圧特性は、非線形の特性であるためこのカーネルを使ってもあまり良い精度が出ないため、使用しなかった。

2.3.3 多項式カーネル

今回、使用を見送ったカーネルについて説明する。

$$K\left(\boldsymbol{x}^{(i)}, \boldsymbol{x}^{(j)}\right) = (1 + \lambda \boldsymbol{x}^{(i)} \boldsymbol{x}^{(j)T})^d \tag{2.29}$$

この関数は、多項式カーネルと呼ばれるカーネル関数である。非線形の特性にも対応しているが、指数関数のような急激な値の変化に弱い特徴がある。MOSFET の電流ー電圧特性は、指数関数的に値が変化することが多く、精度の良いモデルを作ることが不可能であるためガウシアンカーネルを使用することに決めた。

2.4 各種パラメータについてとグリッドサーチについて

2.4.1 パラメータについて

SVR を実行する上で 3 つのパラメータを人間側で決める必要がある。以下に設定が必要なパラメータを示す。

- γ(ガンマ)
- C(コストパラメータ)
- ε(許容誤差)

γ(ガンマ)

γ は、カーネル関数に必要なパラメータである。1 つの訓練データがどの程度モデルに影響を与えるか決めるパラメータである。この値が大きいほど、与える影響は小さくなり、この値が小さいほど与える影響が大きくなる。

C(コストパラメータ)

コストパラメータ C はモデルの複雑度を決めるパラメータである。値が小さいほどモデルは簡単になり、値が大きいほどモデルは複雑化する。

ε(許容誤差)

ε(許容誤差) は、SVR の誤差関数に用いられるパラメータである。$-\varepsilon \leq$ 誤差 $\leq \varepsilon$ のとき、誤差 $= 0$ と扱うため、この値を小さくすれば予測の精度は上がるが、計算時間が増える。逆に、この値を大きくすれば計算時間は減るが、予測の精度は落ちる。

2.4.2　γ と C 関係性について

以下に Figure 2.6 に γ の値とコストパラメータ C による境界の変化を示す。

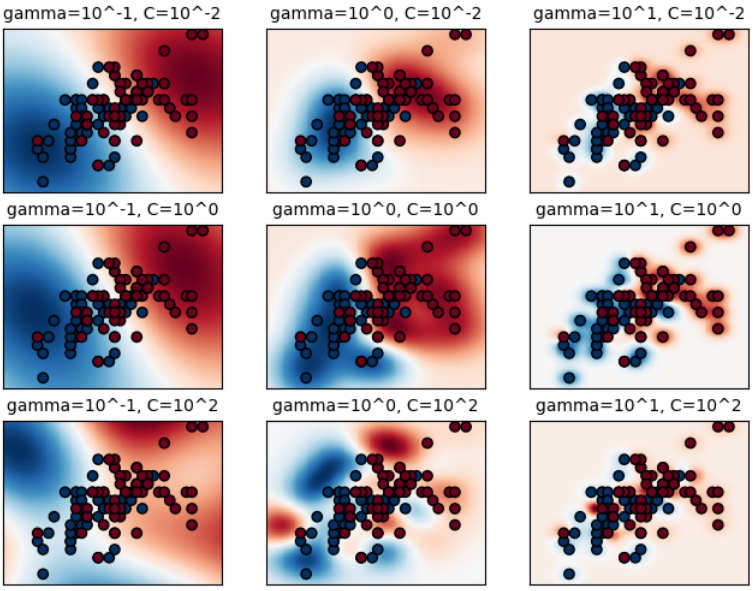

Figure 2.6 γ の値とコストパラメータ C による境界の変化

以上のことから γ は低く、コストパラメータ C は高くある方がよいが、その分時間がかかりトレードオフの関係になっている。そのため、適度な値にする必要がある。

2.4.3　グリッドサーチについて

前節で述べた通り手動で一つ一つ設定してくのは時間がかかる。そこで、グリッドサーチという手法を使う。あらかじめ、パラメータがとりうる範囲を設定し、探索範囲を格子状にして、各格子点でパフォーマンスを評価し最適なものを選ぶ手法である。イメージ図を Figure 2.7 に示す。

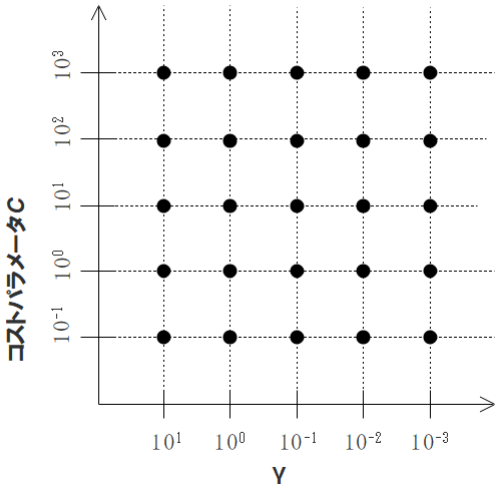

Figure 2.7 gridsearch のイメージ図

このように、それぞれの値の組み合わせをすべて検証し、最も精度のよいモデルになるパラメータを決定する。

第3章　実装方法

実際にどのように scikit-learn の関数を使用しているか説明する。なお、本研究では MOSFET の静特性を機械学習による予測の対象としている。

3.1　訓練データとテストデータのラベル付け

まず、訓練データとテストデータのラベル付けを行う。ラベル付けを行うサンプルコードを code 3.1 に示す。

Code 3.1 label

```
1 practice=pd.read_csv('dataset of practice.csv', header = None, sep = ',')
2 practice.columns=['VD' , 'VG' , 'ID']
3 test =pd.read_csv('dataset of test.csv', header = None , sep = ',')
4 test.columns = ['VD' , 'VG']
```

1,3 行目が csv ファイルの読み込み、2,4 行目がラベル付けである。csv ファイル内は列ごとに区切られており、その左から順に V_D, V_G, I_D を示している。pandas というデータ分析のライブラリを使用し、ファイルの読み込みおよびファイル内のデータのラベル付けを行っている。なお、1,3 行目の header はファイル内の文字の読み飛ばしをするかの確認、sep はファイル内のデータの区切りを指定している。

3.2　グリッドサーチによるパラメータの調整

グリッドサーチによるパラメータの調整を行う。グリッドサーチのサンプルコードを code 3.2 に示す。

Code 3.2 GridSearch

```
1 len = np.arange(start , stop , step)
2 len2 = np.arange(start, stop , step)
3 parameters = [
4     {'C':len,
5      'gamma':len2}
6 ]
7 clf = GridSearchCV(svm.SVR(), parameters ,n_jobs = -1)
8 clf.fit(practice_x , practice_y)
```

pactice __ x: 訓練用データの V_D, V_G の配列 , practice __ y: 訓練用データの I_D の配列

1,2 行目では、numpy という配列管理のライブラリを使用している。arange という等差数列を作成する関数を使っている。7 行目にて、グリッドサーチの設定を行なっている。引数の初項は学習モデルを指定し、第 2 項は 3 6 行目で設定したパラメータの組み合わせを指定する。第 3 項は使用する CPU のコア数を指定でき、−1 した場合は全てのコアを使用する。8 行目で fit() メソッドを使い、パラメータの組み合わせを総当たりで学習する。

3.3 SVR によるモデルの作成

最後に、SVR によるモデルの作成を行う。モデル作成のサンプルコードを code 3.3 に示す。

Code 3.3 Support Vector Regression

```
1 model_lr = svm.SVR(C=clf.best_params_['C'], kernel='rbf', gamma=clf.best_params_['gamma'],
      epsilon=0)
2 model_lr.fit(practice_x, practice_y)
3 y_rbf = model_lr.predict(test_x)
```

test __ x: テスト用データの V_D, V_G の配列

1 行目に SVR の設定を行う。引数の第 1 項ではコストパラメータである C の指定する。前節のグリッドサーチで最もスコアの高い組み合わせを返り値とする best __ params を使用している。第 2 項ではカーネルを指定する。本研究で使用するのは rbf カーネルであるためここでは'rbf' としている。第 3 項は γ の指定である。ここでも第 1 項と同じように best __ params を使用している。第 4 項は ε の指定である。今回は、1.0×10^{-3} で指定している。

2 行目では、fit() メソッドを使い、SVR によるモデルの作成を行う。引数はそれぞれ訓練用データの配列を指定する。そして、3 行目で実際に作成したモデルを使ってテストデータを入力し、その結果を返り値とする predict 関数を使用している。

第4章　実験結果

本章では、作成したプログラムにより、様々な条件の入力データに対して予測を行った結果を紹介していく。以下に様々な条件で予測したときの記録を示す。なお、SVR のパラメータの C と γ はグリッドサーチにて設定し、ε は 1.0×10^{-3} とした。

4.1　仮想データを用いた予測精度の確認

まず、実際のデータを使用する前に仮想のデータを用いて予測できるか確認した。以下に、訓練用データとテスト用データの条件を示し、結果を Figure4.1 に示す。ほぼ重なっているが、○がテスト用データ、×が予測結果である。

・訓練用データ
　V_D: 0〜2.0 V[step: 0.1 V]　　V_G: 0.1〜1.9 V[step: 0.2 V] + 0 V
・テスト用データ
　V_D: 0〜2.0 V[step: 0.1 V]　　V_G: 0〜2.0 V[step: 0.2 V]
・パラメータ
　γ: 0.93　　C: 10.0　　ε: 1.0×10^{-3}

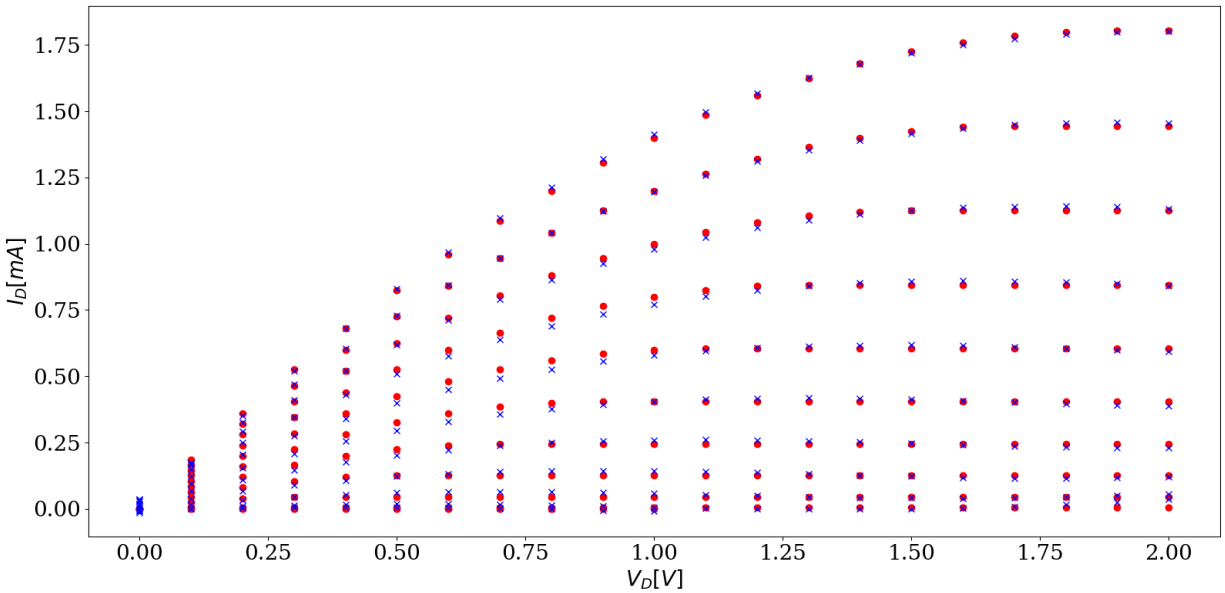

Figure 4.1 仮想データによる予測精度の確認

・最大絶対誤差: 3.74×10^{-2} mA　　・最小絶対誤差: 7.04×10^{-5} mA　　・平均絶対誤差: 1.11×10^{-2} mA

4.2 訓練用 csv ファイルの作成

MOSFET の研究をしている論文を元に作成する。論文に添付されているグラフからプロット点を配置し、csv ファイルとして出力させる。論文の実測データを Figure 4.2 に、抽出されたデータを Figure 4.3 に示す。

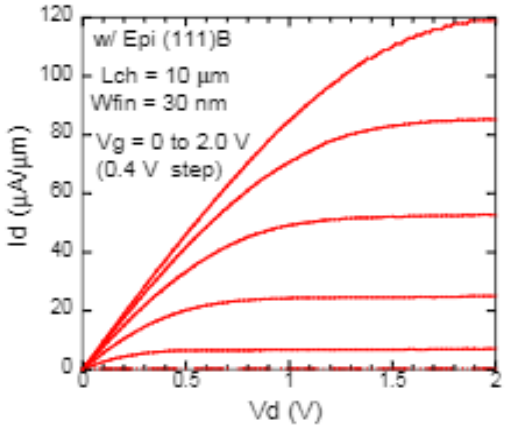

Figure 4.2 論文の実測データ [2]

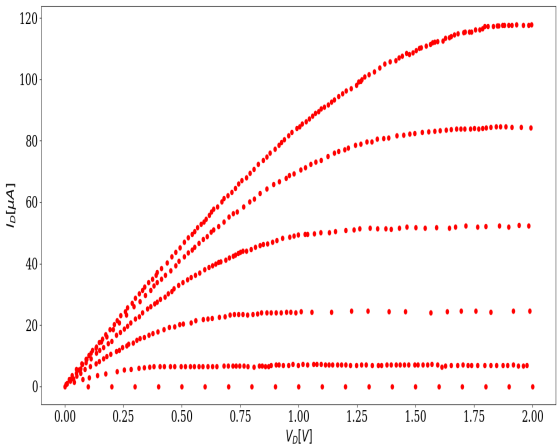

Figure 4.3 論文のグラフからプロット点を抽出した図

以上のように csv ファイルを作成した。プロット点は値が急激に変化する部分は重要な情報であるため多く配置し、それ以外のあまり変化がない場所は適度に配置した。

4.3 元データと学習したモデルの比較

まず、元データから正確なモデルを学習できたか確認した。その図を Figure 4.4 に示す。なお、学習用のデータ数は 484 個である。また、論文より抽出したデータ点を赤丸で、学習によって得られたモデルによる予測データ点を青の×で示す。

・パラメータ

γ: 0.93　　C: 10.0　　ε: 1.0×10^{-3}

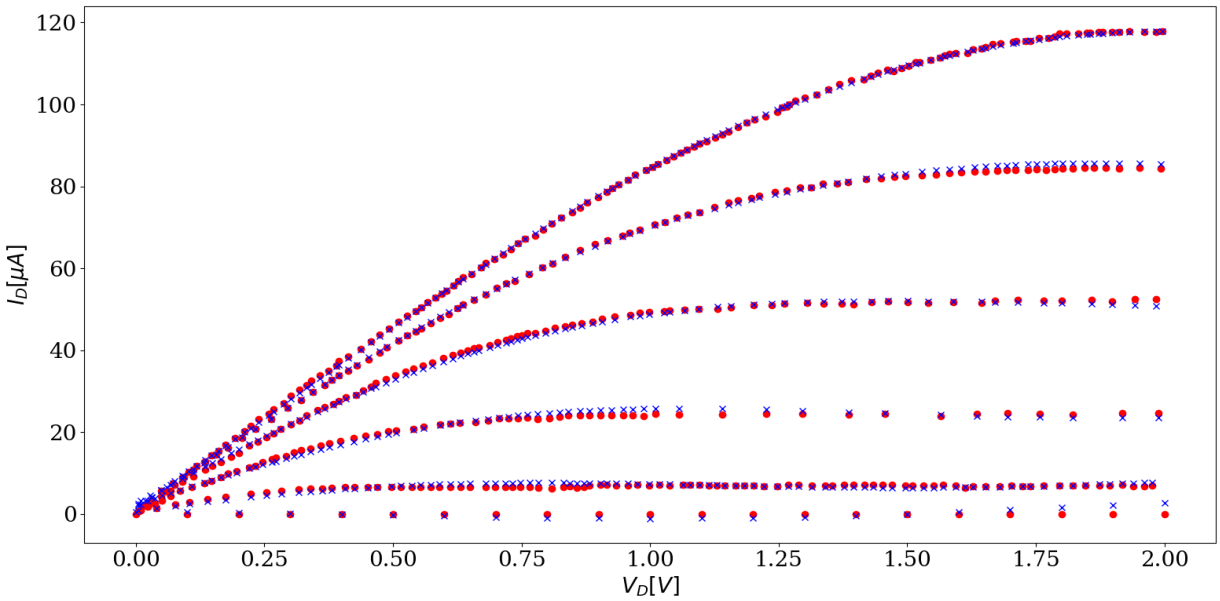

Figure 4.4 元データと学習したモデルの比較

また、誤差は以下の通りになった。

・最大絶対誤差: 2.68 µA　　・最小絶対誤差: 4.89×10^{-4} µA　　・平均絶対誤差: 0.50 µA

4.4 ゲート電圧を変化させたときの比較

学習したモデルを用いて、元のデータには存在しないゲート電圧で予測させた結果を Figure 4.5 に示す。また、V_D=2.0 V での V_G–I_D の値を Table 4.1 に示す。

・パラメータ
　γ: 0.93　　C: 10.0　　ε: 1.0×10^{-3}
・訓練用データ　V_G: 0〜2.0 V[step: 0.4 V]
・テスト用データ　V_G: 0〜2.0 V[step: 0.1 V]

Table 4.1 ゲート電圧を変化させたときの I_D の変化

V_G[V]	I_D[µA]	V_G[V]	I_D[µA]
0.0	2.68	1.1	43.33
0.1	3.13	1.2	50.83
0.2	4.09	1.3	59.09
0.3	5.63	1.4	67.72
0.4	7.79	1.5	76.56
0.5	10.64	1.6	85.45
0.6	14.20	1.7	94.21
0.7	18.50	1.8	102.66
0.8	23.56	1.9	110.60
0.9	29.36	2.0	117.84
1.0	35.87		

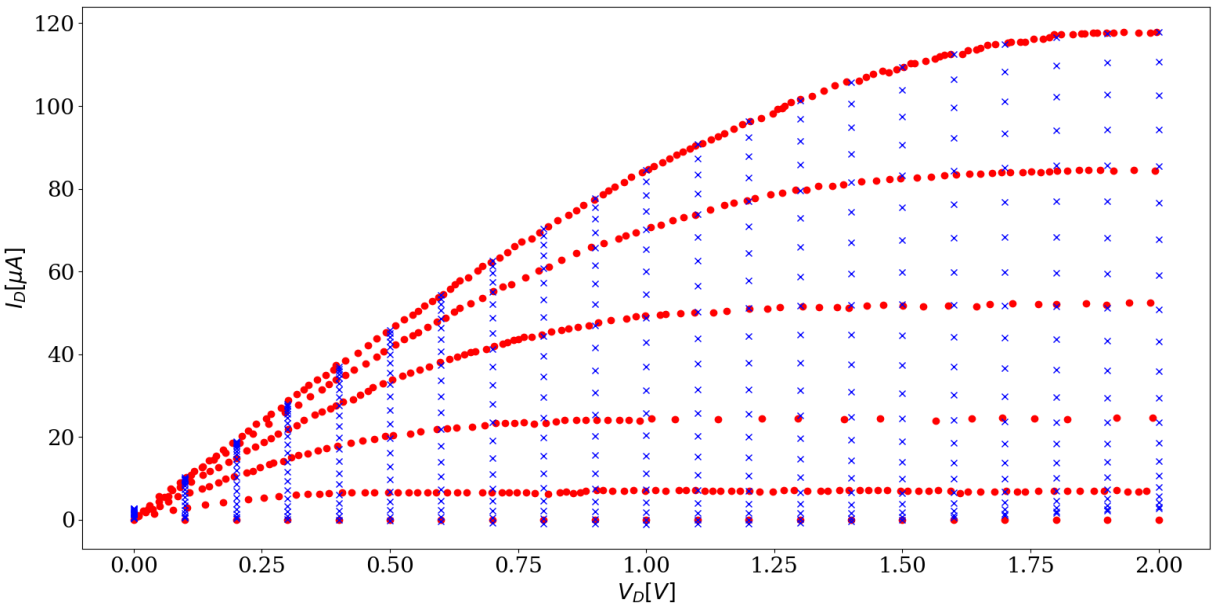

Figure 4.5 ゲート電圧を変化させたときの元データとの比較

4.5 学習モデルの補間の正確さの確認

訓練用データ内のゲート電圧 1.6 V 時のデータを削除したデータで学習させ、元データとどのくらいの誤差が生じたか比較した結果を Fig 4.6 に示す。

・パラメータ
γ: 0.18　　C: 90.0　　ε: 1.0×10^{-3}

・訓練用データ　V_G: 0〜2.0 V[step: 0.4 V] − 1.6 V
・テスト用データ　V_G: 0〜2.0 V[step: 0.4 V]

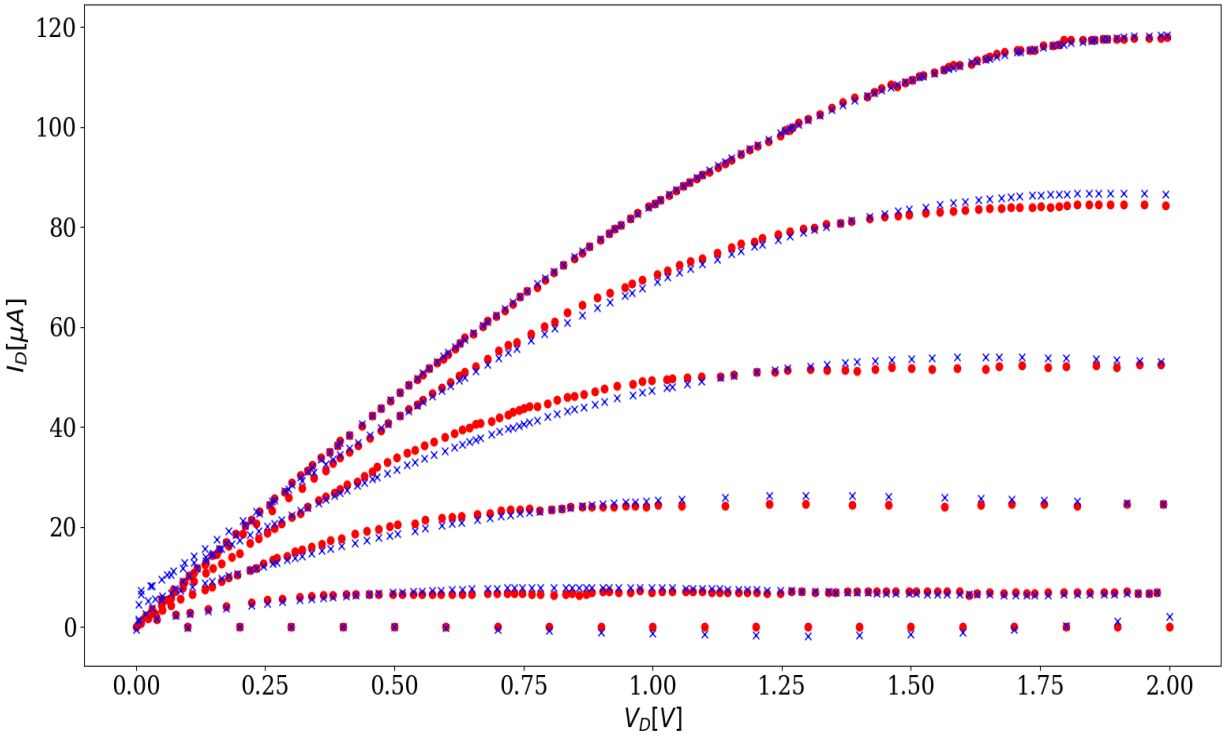

Figure 4.6 V_G=1.2, 1.6V のときのデータを欠落させた状態で学習したときの実測データとの比較

また、誤差は以下の通りになった。

・最大絶対誤差: 6.16 µA　　・最小絶対誤差: 3.92×10^{-4} µA　　・平均絶対誤差: 1.03 µA

つぎに、ゲート電圧が 1.2 V と 1.6 V 時のデータを削除したデータで学習させ、元データとどのくらいの誤差が生じたか比較した結果を Fig 4.7 に示す。

・パラメータ
γ: 0.21　　C: 90.0　　ε: 1.0×10^{-3}

・訓練用データ　V_G: 0〜2.0 V[step: 0.4 V] − 1.6 V , 1.2 V
・テスト用データ　V_G: 0〜2.0 V[step: 0.4 V]

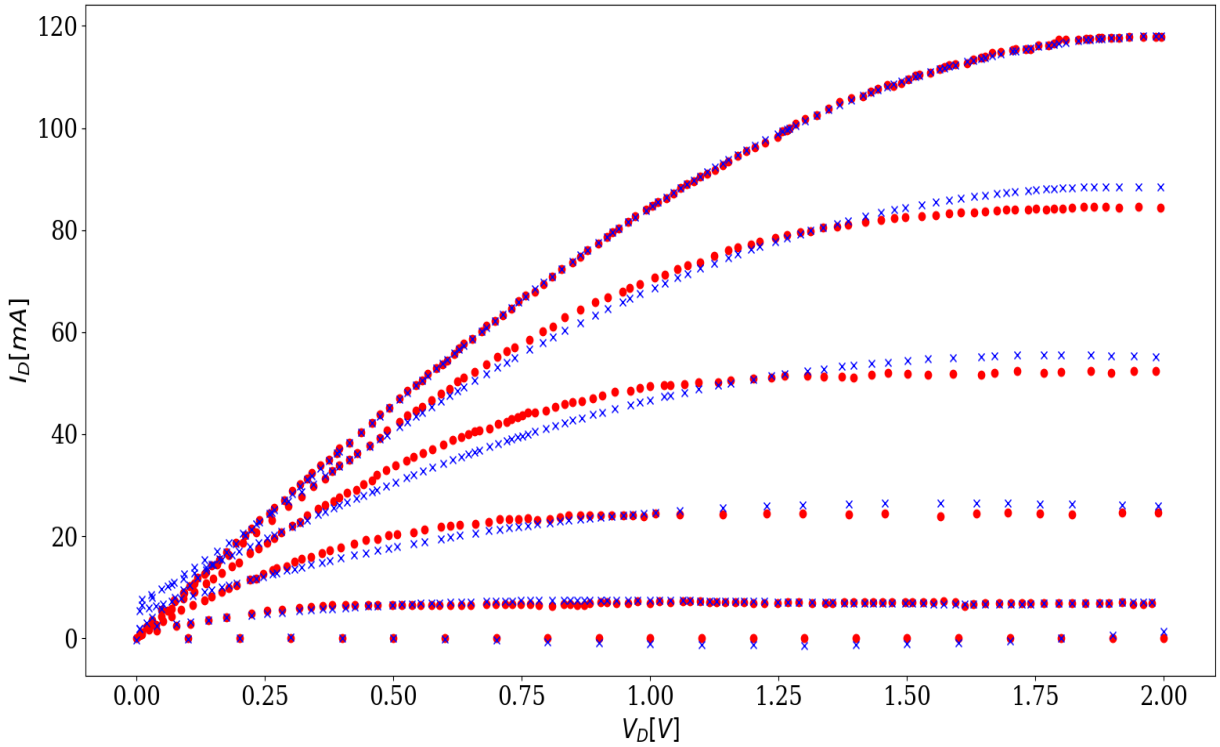

Figure 4.7 V_G=1.6V のときのデータを欠落させた状態で学習したときの実測データとの比較

また、誤差を以下に示す。

・最大絶対誤差: 6.45 µA　　・最小絶対誤差: 4.19×10^{-4} µA　　・平均絶対誤差: 1.23 µA

第5章　考察

5.1　仮想データを用いた予測精度の確認

SVR を用いて作成したモデルによる予測データとテスト用データでは、平均絶対誤差が 1.11×10^{-2} mA であり、最大でも 3.74×10^{-2} mA と概ね大きな誤差なく、正しく予測できていることが確認できた。ただし、$V_D, V_G = 0\,V$ で許容内であるが少々の誤差が出ている。このように、説明変数の数値が 0 のときの予測の精度が落ちている理由としては、説明変数を高次元写像していることやカーネル関数によるものだと考えられる。そのため、電流がマイナス値になるような明らかにエラーが起こっている場合は例外処理などで避けなければならない。このことから、サブスレッショルド特性の予測には難が残ると推測される一方、ON 状態での性能予測においては、V_D のレンジは 0.5 V から 2.0 V で使われることが多く、大きな問題にはならないと考えられる。

5.2　元データと学習したモデルの比較について

予測データと論文から得られたデータと大きな誤差はなく平均絶対誤差も 0.50 µA であり、最大でも 2.68 µA で収まっており、概ね正しいモデルを作成できていることが分かる。グリッドサーチによるパラメータの調整がうまく機能していることが確認できる。

5.3　ゲート電圧を変化させたときの比較について

Table 4.1 からも分かる通り、I_D の V_D 依存性を正しく表現するとともに、V_G が増えると、その α 乗 ($1 \leqq \alpha \leqq 2$) に比例して I_D が上昇する非線形的変化を正しく学習できている。また、論文から得られたデータと比較しても目立った誤差はないと考えられる。

5.4　学習モデルの補間の正確さの確認について

1.6 V の論文から得られたデータがない場合のモデルは、全体的に予測データがずれていることが分かる。特に、1.6 V と値が近い 1.2 V のときが影響を受けていることが分かる。最大誤差も 6.16 µA であり、実験結果 4.3 より 2 µA ほど増加した。平均誤差も 1.03 µA と増加しており、約 2 倍程度の誤差が生じていることが分かる。ただし、$V_G = 2.0\,V$ での予測データは影響をほとんど受けていないため、γ などのパラメータを調整するよりも回帰アルゴリズム自体に手を加える必要がある。1.2 V と 1.6 V の論文から得られたデータがない場合のモデルは、さらに精度が悪くなっている。しかし、1.6 V の論文から得られたデータがないときの実験結果と比べて多少の誤差に収まった。本来であれば、約 2 倍程度の誤差が生じるはずだが、最大誤差でも 6.45 µA で収まっていることから、回帰アルゴリズムは、上下の振れ幅さえ決まっていればその範囲での予測はある程度の情報があれば正確に予測することが可能ではないかと推測できる。

また、どちらの結果でも $V_D, V_G = 0\,V$ のときに、論文から得られたデータとかけ離れている。しかし、V_D のレンジは 0.5 V から 2.0 V で使われることが多く、大きな問題にはならないと考えられる。

第6章　結論と今後の課題

6.1　結論

本研究では、機械学習を用いた V_G–V_D–I_D 特性を予測することを目的としていた。実験結果から見て取れるように、概ね目的は達成されたと思われる。

仮想データや論文から得られたデータから作成したモデルを使用した予測では、サブスレッショルド特性の予測に難はあるものの、許容範囲内の誤差で収まっており、良い結果が得られたと考えられる。
また、それらのモデルを使用した予測では、I_D の V_D 依存性を正しく表現するとともに、V_G が増えると、I_D が上昇する非線形的変化を正しく学習できていると推測できる。
最後に、学習モデルの補間の正確さは、一定量情報が減ると、精度は下がるもののそこからは精度があまり減少しないことから、試作したデバイスから得られた情報が少なくてもある程度の予測精度を持ったモデルを作成できることが分かった。つまり、測定者が大量のデータをとる必要がなくなるため、デバイスやコンパクトモデルの作成者だけでなく、測定者に対しても負担が軽減できることが期待できる。

6.2　今後の課題

まず、精度の向上があげられる。とくに、デバイスは大量に試作品を作ることが困難であるため、必然的にデータ量は少なくなる。そのため、データ量が少なくても精度が落ちないようにする必要がある。グリッドサーチと別の手法を一緒に使用してパラメータの調整をしたり、全く新しいカーネルを開発するなどすれば精度が向上できると考えられる。

次に、サブスレッショルド特性の予測についてである。現状、どのパラメータでも改善しなかったため、SVR を応用した回帰アルゴリズムを作成することが、最善策だと予想される。

最後に、ゲート酸化膜やチャネル長や誘電率などの説明変数の追加である。本研究では、V_G と V_D, I_D しか使用しなかったが、誘電率やゲート酸化膜などを使用することで、未知のデバイスに対しての予測が可能になりうる。

参考文献

[1] T. Irisawa et al., ”High Electron Mobility Triangular InGaAs-OI nMOSFETs with (111)B Side Surfaces Formed by MOVPE Growth on Narrow Fin Structures”, 2013 IEEE IEDM (2013).

[2] A. Mostefai, et al., ”Modeling and simulation of transistor mosfet(high-k)using neural network”, Journal of Electricla Engineering, Volume 17(2017).

[3] ”Choosing the right estimator”, 閲覧日: 2022 年 2 月 22 日.
https://scikit-learn.org/stable/tutorial/machine_learning_map/

[4] ”サポートベクターマシン (Support Vector Machine, SVM)”, 閲覧日: 2022 年 2 月 22 日.
https://datachemeng.com/supportvectormachine/

[5] ”サポートベクター回帰 (Support Vector Regression, SVR)”, 閲覧日: 2022 年 2 月 22 日.
https://datachemeng.com/supportvectorregression/

謝辞

本研究の遂行にあたり、研究内容や論文執筆まで適切な助言を賜った、木場先生に心から感謝いたします。自分の興味のある内容で研究することができました。また、文谷君、山﨑君にも相談に乗っていただいて研究を順調に進むことができました。ありがとうございました。